

# tmux documentation

## tmx-ssh-install

**Revision:** 1 **Last modified:** 2026-05-22T00:00:00Z **Authority:** vasic-digital tmux project  
**Maintainer:** milosvasic **Scope:** §11.4.18 script companion doc for `scripts/tmx-ssh-install.sh`

### Purpose

Client-side, idempotent bootstrapper for the `ssh <host>-tmx <session> ergonomics` described in design spec docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md §4.D and §5.2. It generates a per-host ed25519 key, installs the matching `command=".../tmx-ssh-dispatch.sh"` entry in the remote `~/.ssh/authorized_keys`, writes a local `Host <host>-tmx` alias to `~/.ssh/config`, and verifies the dispatcher rejects unknown commands as designed. Every step is a no-op on re-run, so the script is safe to use as part of an automated setup loop.

### Usage

```
bash scripts/tmx-ssh-install.sh <user>@<host> [--remote-project-path /path]
bash scripts/tmx-ssh-install.sh --uninstall <user>@<host>
bash scripts/tmx-ssh-install.sh --purge-key
bash scripts/tmx-ssh-install.sh --dry-run <user>@<host> [--remote-project-path /path]
bash scripts/tmx-ssh-install.sh --help
```

Defaults:

- `--remote-project-path` → `~/Projects/tmux` (matches the operator's local convention; override when the remote checkout lives elsewhere).
- `--force` → overwrite an existing per-host key. **Default is to leave the key in place** so re-runs do not invalidate prior installs.

### Inputs

#### Positional

- `<user>@<host>` — the SSH target. Required. Must match `^[A-Za-z0-9._@-]+$`; rejected otherwise.

#### Flags

| Flag                                    | Effect                                                                                          |
|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>--remote-project-path PATH</code> | Where the vasic-digital tmux clone lives on the remote (default <code>~/Projects/tmux</code> ). |

| Flag                     | Effect                                                                                                  |
|--------------------------|---------------------------------------------------------------------------------------------------------|
| <code>--uninstall</code> | Reverse mode. Removes the <code>authorized_keys</code> entry (by fingerprint) and the local Host block. |
| <code>--purge-key</code> | Only with <code>--uninstall</code> . Also removes the local per-host key.                               |
| <code>--force</code>     | Overwrite an existing local per-host key. Loud and intentional.                                         |
| <code>--dry-run</code>   | Print every action without executing it. Safe to run anywhere.                                          |
| <code>-h / --help</code> | Print usage and exit 0.                                                                                 |

## Environment

No environment variables are read directly. SSH itself honours `SSH_AUTH_SOCK`, etc. — pre-existing auth must work for step 3 (reachability probe).

## Outputs

### Files modified (install)

- Local: `~/.ssh/id_tmx_<sanitized-host> + .pub` (created if absent).
- Local: `~/.ssh/config` (appended `Host <host>-tmx` block — once).
- Remote: `<remote-project>/scripts/tmx-ssh-dispatch.sh.template` (scp'd).
- Remote: `<remote-project>/scripts/tmx-ssh-dispatch.sh` (substituted + mode 0755).
- Remote: `~/.ssh/authorized_keys` (appended `command="..."` line — once per fingerprint).

### Files modified (uninstall)

- Local: `~/.ssh/config` (drops the matching Host block).
- Remote: `~/.ssh/authorized_keys` (drops the line whose pubkey fingerprint matches the local pubkey).
- Local with `--purge-key`: `~/.ssh/id_tmx_<sanitized-host> + .pub`.

## Stderr stream

Every step logs `[tmx-ssh-install] step N: ...` so the operator sees exactly which action is in flight. Errors are prefixed `[tmx-ssh-install] ERROR:.`

## Side-effects

- Generates a new SSH key only if one does not already exist for the target host. Never overwrites without `--force`.
- Touches the remote's `~/.ssh` (mode 700) and `authorized_keys` (mode 600) to keep `sshd` happy.
- Performs a single SSH probe to assert reachability before mutating anything remote.

## Dependencies

- POSIX `/bin/sh` (script runs under macOS bash 3.2 and any Linux dash/bash).

- `ssh`, `scp`, `ssh-keygen`, `awk`, `mktemp`, `tr`, `sed`, `grep`.
- Pre-existing working SSH auth path to the target (used to install our key). If non-interactive `ssh -o BatchMode=yes <target> exit 0` fails, the installer aborts and tells the operator to run `ssh-copy-id` first.

## Cross-references

- `scripts/tmx-ssh-dispatch.sh.template` — the remote-side script this installer deploys; see `docs/scripts/tmx-ssh-dispatch.md`.
- `scripts/tmx-shell-init.sh.template` — the interactive shell flow the dispatcher hands off to on empty `SSH_ORIGINAL_COMMAND`.
- `scripts/tmx-state/main.go` — the cwd-recall daemon the dispatcher queries.
- Design spec: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md` §4.D, §5.2, §6 (edge cases 9 + 13).
- Governance: §11.4.18 (script docs), §11.4.44 (revision header), §11.4.67 (POSIX-parseable shell), §11.4.10 (never leak credentials — this script never reads existing private keys).

## Last verified

2026-05-22 — `sh -n clean, --dry-run` exercises all 8 install steps and all 3 uninstall steps without performing any I/O. Full real-host verification deferred to P9 release verification per spec §10.